Deep Learning Digit Classification Project Report

Handwritten Digit Classification using TensorFlow and MNIST Dataset

Abstract

This project implements a Deep Learning model using TensorFlow and Keras for handwritten digit classification. The MNIST dataset was used to train and evaluate the model. The objective was to develop a neural network capable of recognizing handwritten digits from 0 to 9. The model was trained using a feedforward neural network architecture and achieved high classification accuracy on the test dataset.

1. Introduction

Deep Learning is a subset of Artificial Intelligence that enables computers to learn patterns from large amounts of data. Image classification is one of the most common applications of deep learning.

In this project, a neural network was developed to classify handwritten digits using the MNIST dataset. The model learns patterns from pixel values and predicts the corresponding digit.

2. Objective

The objectives of this project are:

- To understand image classification using deep learning.
- To build a neural network using TensorFlow and Keras.
- To train and evaluate the model using the MNIST dataset.
- To visualize training performance using graphs.
- To save the trained model for future predictions.

3. Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits.

Dataset Details:

- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28 × 28 pixels
- Number of Classes: 10 (Digits 0–9)

The dataset is widely used for evaluating machine learning and deep learning algorithms.

Training Data: (60000, 28, 28)

Testing Data: (10000, 28, 28)

4. Data Preprocessing

Data preprocessing was performed before training the model.

1. Loaded the MNIST dataset.
2. Normalized pixel values by dividing by 255.
3. Converted image values into a range between 0 and 1.

Normalization improves model performance and training efficiency.

5. Sample Image Visualization

A sample handwritten digit image was displayed to understand the dataset structure.

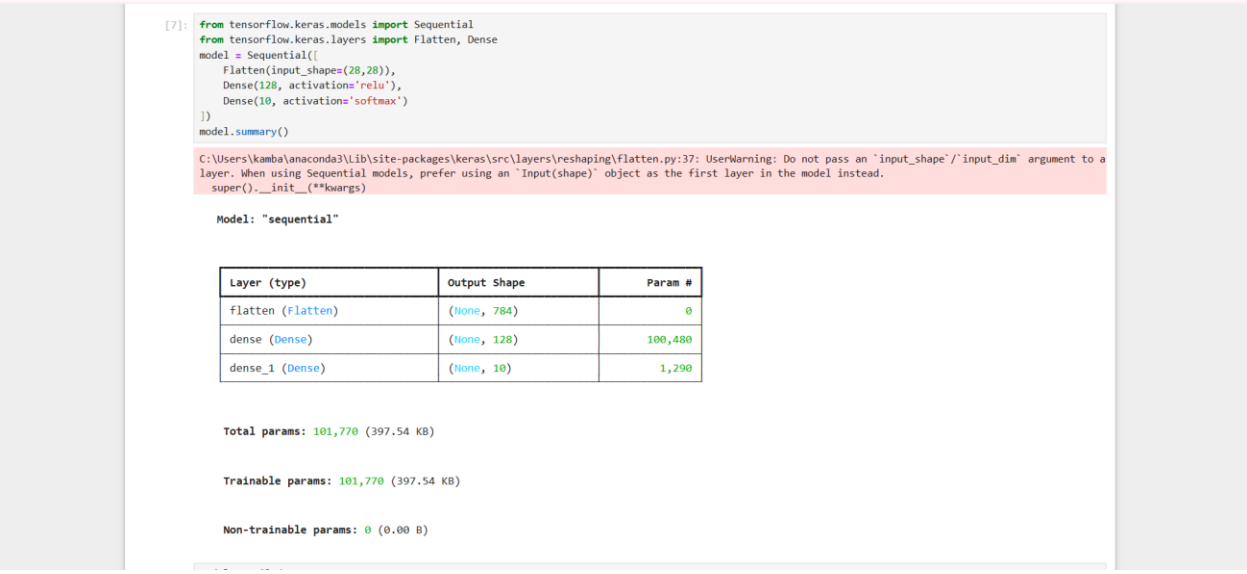
6. Model Architecture

A Sequential Neural Network model was created using TensorFlow Keras.

Architecture:

1. Flatten Layer
2. Dense Layer (128 neurons, ReLU activation)
3. Dense Output Layer (10 neurons, Softmax activation)

The model contains 101,770 trainable parameters.



```
[7]: from tensorflow.keras.models import Sequential
     from tensorflow.keras.layers import Flatten, Dense
     model = Sequential([
         Flatten(input_shape=(28,28)),
         Dense(128, activation='relu'),
         Dense(10, activation='softmax')
     ])
     model.summary()
```

```
C:\Users\kamba\anaconda3\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

7. Model Compilation

The model was compiled using:

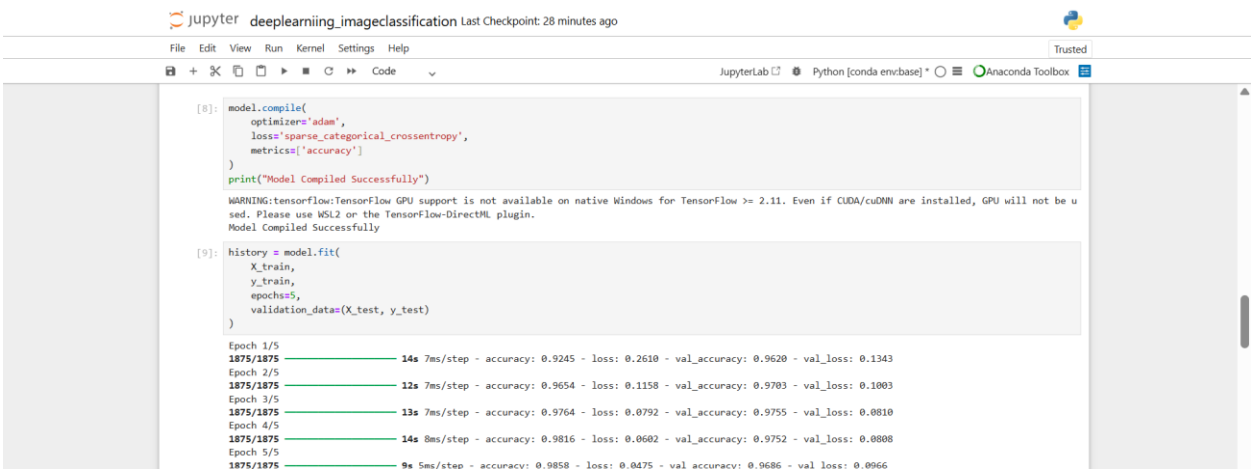
- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Evaluation Metric: Accuracy

The Adam optimizer was selected due to its efficiency and fast convergence.

8. Model Training

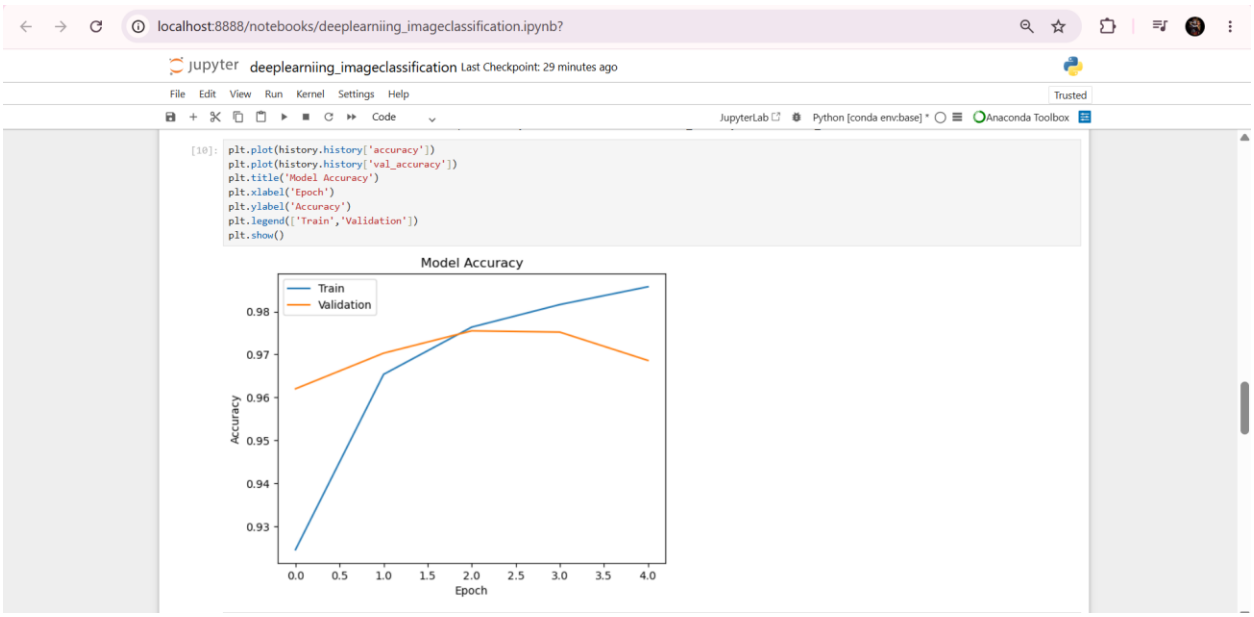
The model was trained using the training dataset for 5 epochs.

During training, the model continuously improved its accuracy by learning patterns from handwritten digit images.



9. Training Accuracy Curve

The training and validation accuracy curves were plotted to observe model learning performance.



Observation

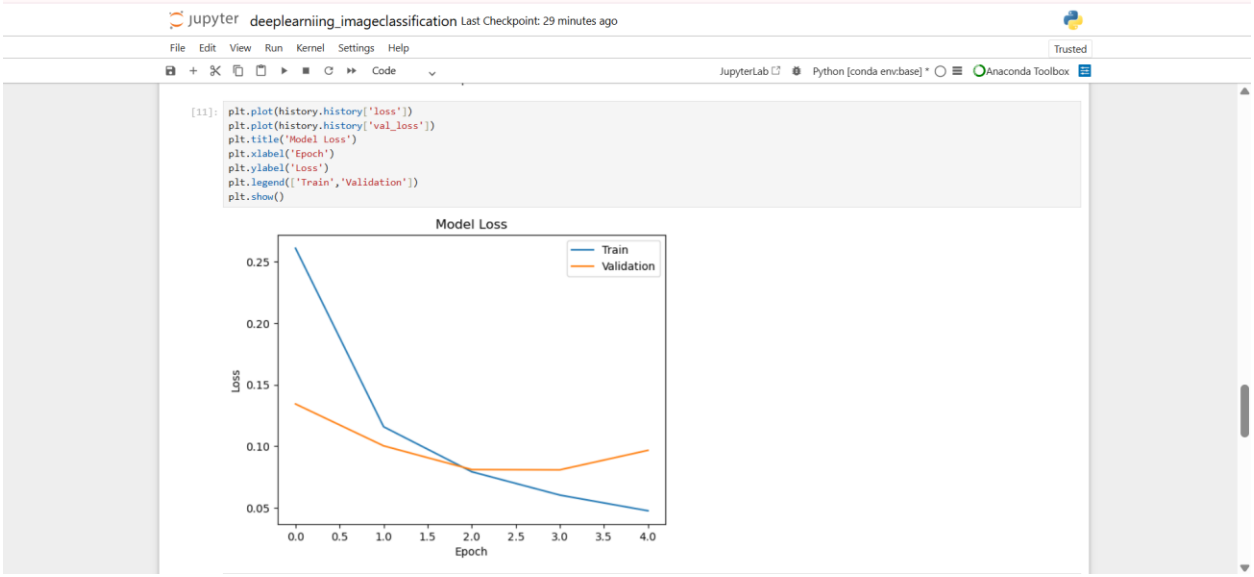
The accuracy increased steadily with each epoch, indicating successful learning.

10. Training Loss Curve

The training and validation loss curves were plotted to monitor model error.

Observation

The loss decreased as training progressed, indicating model improvement.

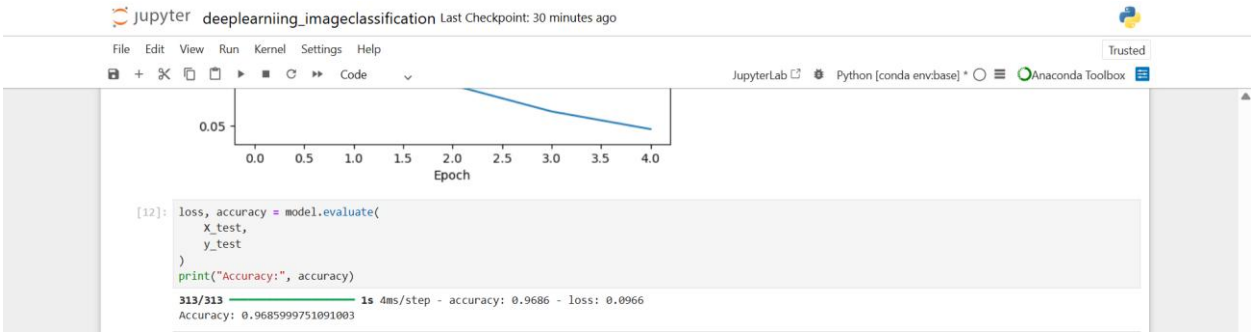


11. Model Evaluation

The trained model was evaluated using the testing dataset.

Evaluation Metric:

- Accuracy



Observation

The model achieved high classification accuracy on unseen test data.

12. Model Saving

The trained model was saved using:

digit_classifier_model.h5

Saving the model allows future predictions without retraining.

```
[13]: model.save("digit_classifier_model.h5")
      print("Model Saved Successfully")

      WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We
      recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
      Model Saved Successfully
```

13. Inference and Prediction

The saved model was used to predict handwritten digits from the test dataset.

Example Prediction:

Predicted Digit: 7

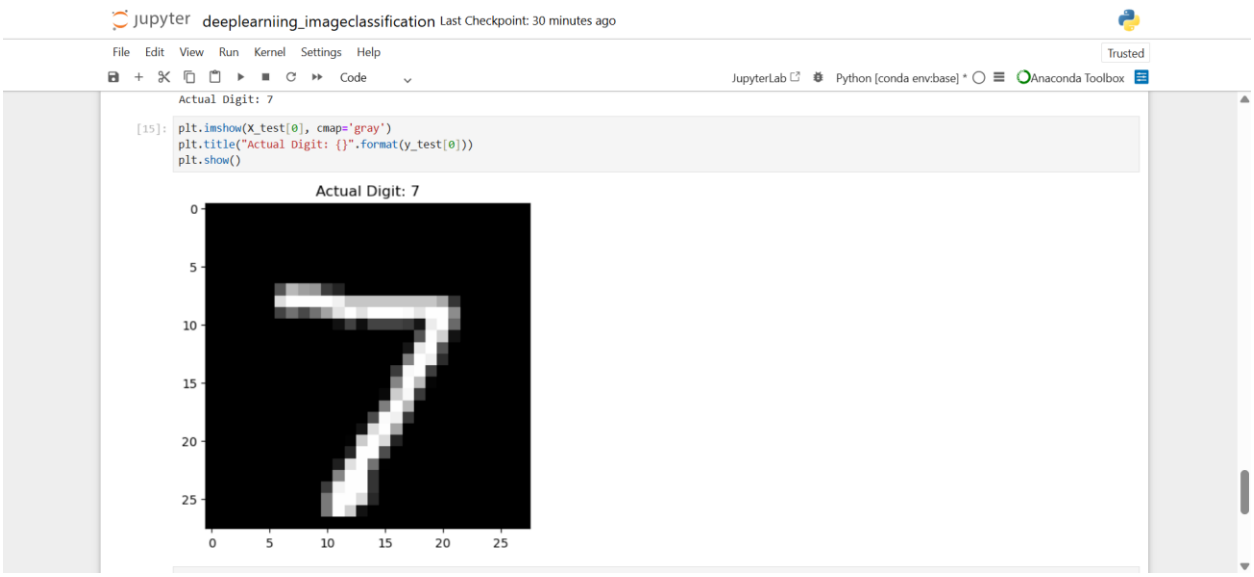
Actual Digit: 7

```
[14]: prediction = model.predict(X_test[:1])
      import numpy as np
      print("Predicted Digit:", np.argmax(prediction))
      print("Actual Digit:", y_test[0])

      1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step
      Predicted Digit: 7
      Actual Digit: 7
```

Observation

The predicted digit matched the actual digit, demonstrating successful classification.



14. Results

The neural network successfully classified handwritten digits with high accuracy.

Key Achievements:

- Successfully loaded and preprocessed the dataset.
- Built a neural network using TensorFlow.
- Trained and evaluated the model.
- Generated training accuracy and loss graphs.
- Saved the trained model.
- Performed successful predictions.

15. Conclusion

This project successfully implemented a Deep Learning-based image classification system using the MNIST dataset.

The model demonstrated excellent performance in recognizing handwritten digits. Training and validation curves indicated effective learning, and the final evaluation accuracy confirmed that the model generalized well to unseen data.

The trained model was saved and used for inference, making it suitable for future digit recognition tasks.

References

1. TensorFlow Documentation: https://www.tensorflow.org
2. Keras Documentation: https://keras.io
3. MNIST Dataset: http://yann.lecun.com/exdb/mnist/
4. NumPy Documentation: https://numpy.org
5. Matplotlib Documentation: https://matplotlib.org